K215: Checked Affine Cores and a Completed CM215 Endpoint

Research and verification packet for Vladimir Reshetnikov's ProveIt

21 October 2026

Abstract

We study proof simplification and further state reduction for the repository's 215-state binary Turing machines for ZFC consistency. The principal new proof artifact closes a gap in the existing affine-core proposal: it proves, in Coq, that both literal register programs `p215` and `dagPerm` halt exactly when the same 367-vertex accelerated graph halts, for every natural-valued register file. The construction recognizes 1,059 drains in the 1,426-vertex checkpoint graph, proves their exact arithmetic behavior once, checks all concrete source bindings, and preserves halting in both directions. The theorems require no added axioms. This is an auxiliary proof reduction, not a reduction of the binary machine's 215 working states.

The packet also completes verification work for the existing CM215 transport proposal, checks an inherited Lean proof-shortening patch, and supplies a new search using a postcondition that permits spare tape data to change. No verified machine with 214 or fewer states is claimed. The search records distinguish bounded screening, explicit counterexamples to primitive contracts, and actual proof-assistant theorems. All prior machine designs, proposals and calibration results retain their attribution.

Contents

1 Findings, baseline and scope 2
2 A proof architecture that separates the right obligations 3
3 The exact checkpoint semantics 5
4 The total drain theorem 5
5 Contracting drains without losing divergence 6
6 Binding the 367-vertex core to both literal programs 7
7 Which other 215-state machine is easiest to prove? 8
8 A direct opportunity to shorten the Lean semantic proof 9
9 Research toward 214 states 10
10 Integration plan 13
11 Further research questions 14
12 Reproduction and provenance 14
13 Primary sources 15

1 Findings, baseline and scope

The repository baseline for this packet is `56e487ce8fa4cf63ad169ffe0cd2944e9fc62dd3`. The live repository changed while the review began; this fixed revision, together with recorded Git blob IDs, SHA-256 hashes and explicitly recorded proof patches, defines what was actually checked. We reviewed both the maintained proof lanes and the newly placed proposal packets. The incoming quotient-detachment packet was also read before choosing the new work. The primary sources are listed in Section 13.

Three distinctions are essential when evaluating progress in this program. First, a shorter proof *interface* need not be a shorter complete build. Second, a smaller register-program graph need not compile to fewer binary Turing-machine states. Third, a candidate that fails a uniform primitive simulation contract has not thereby been shown to have the wrong blank-tape consistency behavior: the failing frame might not occur in that particular blank run. The packet uses these distinctions throughout.

1.1 The existing exact endpoint

The primary K215 machine has 215 working states, labels $0, \ldots, 214$, a separate halt label 215, and 430 deterministic transition rows. It uses one two-way infinite binary tape, blank symbol 0, initial head position 0, and only left/right moves. Its established Lean endpoint is

```
KEndpoint215.blank_halts_iff_zfc_refutation :
  BlankHalts machine215 <-> BProv ZFCax_s [] Form.fBot
```

The companion theorem negates both sides to obtain syntactic consistency. The Rocq lane has the analogous exact dense-machine endpoint. These are external checked equivalences with finite ZFC refutability. They do not prove unconditional nonhalting or the consistency of ZFC, and they are not a new construction of an internally encoded PA or ZFC proof object. The ordinary argument that the constructions and equivalence can be formalized in ZFC is inherited from the project's calibration work.

1.2 What was already available

The current proposals already contain several improvements that should not be advertised again as new results: a single checked certificate plus a generic mirror theorem; quotienting addresses by their trailing zero suffix; reducing 2,240 checkpoint entries to 1,426; balanced certificate lookup; automatic certificate inference from literal trees; shared decoder contexts; concrete zero-credit parser and prefix-credit theorems; and local 215-state transport proofs. The quotient-detachment packet already proves a larger numerical inference rule equiconsistent with ZFC, but reports no smaller physical machine. The existing affine-core packet already identified the 1,059 drains, the 367-vertex graph and 33 shared update matrices. Its generic drain theorem, source-to-graph binding and complete contraction proof were not kernel checked there. Those are the main gaps closed here.

The maintained Lean source proof already uses a balanced `CTree` certificate lookup and staged concrete kernel checks. The earlier balanced lookup proposal addresses the Rocq flat-list checker; it should not be described as a missing Lean optimization. The existing Lean automatic-certificate proposal is a different improvement: infer the relation from the literal trees, then check a normalized 1,426-entry certificate with shared tree bindings. Attaching that existing facade to the complete endpoint is a useful separate integration task. Neither that proposal nor the timings in this packet establish which operation dominates a full Lean build.

1.3 Verification record

The accompanying component receipts contain exact commands, compiler versions, source hashes, module outcomes and assumption reports. The following inserted record is generated from the final accepted runs, rather than from a planned test count.

- Coq 8.18.0 compiles all seventeen source-lane modules, including the exact two literal source/-core equivalences, same-name public facade and typed client. All twenty-five printed assumption reports are closed. A separate recursive `coqchk EndpointClient` passes with exit code zero. Compilation takes 92.585 seconds and the recursive check 440.022 seconds in the recorded run; these are reproduction observations, not comparative speed claims.
- Lean 4.32.0 accepts the complete 184-module K215 project closure and both original typed endpoint audits. This baseline contains 180 byte-identical project modules and four disclosed certificate-proof batching overlays. A subsequent verified resume preserves the source, inventory, toolchain and proof receipts and performs zero new compiler attempts.
- The exact CM215 attachment passes all eight modules: seven inherited modules and an independent typed client. The client checks finite-run halting and its negation against the concrete machine. All printed dependency axioms are among `propext`, `Classical.choice` and `Quot.sound`.
- The unchanged inherited `KSeedModel222` patch passes all ten affected project modules and four endpoint/audit clients, including K215 and CM215 endpoints. With this patch and the batching overlays together, 179 of the 184 project modules remain byte-identical to the pin and five have disclosed proof changes. Printed axioms remain within the same standard set.
- Independent finite audits replay all 59 survivors of the relaxed-spare screen, all 639 coordinated-provider cases and parent controls, and the three detailed startup/prefix checks. Their distinct failure and timeout categories are reported in Section 9. The separate symbolic verifier accepts all 1,900 increment occurrences for the rejected no-append table; this is quantified symbolic evidence outside a proof-assistant kernel.

2 A proof architecture that separates the right obligations

The most useful long-term structure has three semantic interfaces. The first relates finite derivations in the numerical calculus to finite ZFC refutations. The second relates the literal register program to a compact graph of tested operations and total arithmetic macros. The third relates that program, or a previously proved exact machine, to the literal binary transition table. Each interface has different optimization opportunities.

At the logical interface, use one interpretation/substitution theorem and derive special cases by composition. At the register-program interface, compute finite correspondence data from literal syntax and validate it in the kernel. At the machine interface, use local invariants and positive finite simulation blocks when the new table is a small modification of a proved table. Rebuilding the calculus or the whole register runtime for such a local table change adds avoidable work.

A concrete dependency cleanup is available in the source imports. `Prog215.v` uses `K230.Prog230` only for its small generic `cedge` datatype, constructors and `cv` decoder; it does not reference `p230`. That inherited module contains 5,166 lines and took 19.1 seconds in the focused build recorded here. Extracting the generic codec into a shared module, updating generated imports and preserving required public names would remove the unrelated literal 230-state program from this source closure.

This is a concrete follow-up proposal, not an applied patch or a measured saving for the full endpoint. The current proof deliberately retains byte-identical production program inputs.

The Lean build also motivates a resource-oriented proof change. The original monolithic `fwd_ok12` check terminated with exit code 137 under an 8 GiB memory limit, without a timeout. The recorded overlay replaces the proofs in certificate chunks 12–15: each 140-entry check becomes four private 35-entry `decide +kernel` lemmas, followed by checked Boolean reassembly. Imports, public statements, program data and certificate data are unchanged. The companion generator patch emits exactly the four checked replacements and leaves the other fourteen partitioned source modules byte-identical. This applies the quarter-block technique already present in the earlier normalized and automatic-certificate proposals. The source exceptions and the failed original attempt are preserved explicitly. This adjustment makes the checking work smaller per lemma; the four modules grow from 11 to 36 lines each, adding 100 generated lines, and must not be counted as part of the separate handwritten line reduction in `KSeedModel222`.

For the two current register programs, normalized checkpoint correspondence already gives a particularly direct proof. Our affine construction should not automatically replace that fast path. Its main benefit is a reusable semantic interface for future sources with different implementations of transfer, copy or clearing loops. Such programs can have unequal checkpoint graphs while performing the same total affine operations. A proof about their arithmetic cores can then avoid repeating hundreds of low-level loop inductions.

2.1 The next substantial runtime-proof reduction

The most promising next reduction in the large runtime proof is to complete the attachment of the existing framed decoder component. That earlier packet already supplies an arbitrary-tape framing theorem and checks 281 local blocks over 234 decoder roles, with 97 exposed primitive-entry contexts. These are shared internal traversals of the source DAG. They are not yet 97 complete primitive contracts or a replacement for the 4,751-occurrence site-coverage proof. The missing attachment has three concrete parts.

1. Bind the decoder's source DAG expansion to the literal `KSource215` program inside the proof assistant. The existing pilot has an independent Python expansion audit, which is useful evidence but does not supply this kernel equality.

2. Prove by induction on a valid source path that decoder roles produce the framed block-chain premises. Carry the exact head location and preceding bits, including the single left marker at the root. Then invoke the existing arbitrary-tape theorem, so the unexamined gap, header, registers and spare suffix are preserved without repeating whole-tape calculations at each site.

3. Prove the exposed primitive and return contracts with a variable code prefix, the exact prefix-language restriction, arbitrary register data and the appropriate zero/nonzero guard. Factor carry and strip behavior into list-induction lemmas, compose the existing register-box contracts, and recover the same public site-coverage theorem.

The prefix-language condition is essential. Two preceding bits suffice for the already checked internal decoder blocks; they have not been proved sufficient for arbitrary leaf actions and pointer returns. Break and zero-decrement returns can overflow the code prefix at particular source sites, so the marker movement and ancestor counts must remain in the contract. Likewise, a shared physical state alone is not a sound decoder role: the source DAG node is part of its meaning. These constraints explain precisely what must be proved before the existing local pilot can simplify the full runtime.

This is a proposed integration of prior work, not a completed replacement in this packet. It has greater potential to remove repeated runtime proof work than merely lowering source-tree depth, but its effect on kernel memory and clean-build time must be measured after the exact endpoint is reattached. The current packet retains and checks the full site-coverage dependency chain.

3 The exact checkpoint semantics

A register file is a function $R : \mathbb{N} \to \mathbb{N}$. Although the concrete program uses nine registers, the generic theorems quantify over every natural-valued register file. Unused registers are preserved. Pointwise equality

$$R \equiv S \quad \Longleftrightarrow \quad (\forall i \in \mathbb{N})\ R_i = S_i$$

is carried explicitly, so functional extensionality is not an added axiom.

Each graph vertex is either a halt or a test

$$v : \mathrm{Test}(r, (u, a), (w, b)),$$

where a and b are finitely supported nonnegative increment vectors. If $R_r > 0$, the graph first decrements r, then adds a, and continues at u. If $R_r = 0$, it adds b and continues at w. The order of the decrement and the following arithmetic update is part of the semantics. Omitting that decrement would invalidate the contraction even when every target vertex was correct.

The source-to-graph checker verifies the literal instruction at each graph address, the register being tested, both branch directions, every increment count between tests, and the next source address modulo trailing zeros. It uses the inherited segment executor, which is already proved to execute only increments and control instructions until the next test or halt. A finite graph-closure check guarantees that each target is defined.

The checker does not trust the Python graph generator, an address hash, or a claim that two trees look alike. Its concrete acceptance theorems are evaluated against the literal `Prog215.p215` and `DagPerm.dagPerm` definitions. Their bytes match the maintained production files at the pinned revision. The generated data is input to a proof, not an extra axiom.

4 The total drain theorem

Definition 4.1. A drain at v has the shape

$$v : \mathrm{Test}(r, (v, a), (w, b)), \qquad a_r = 0.$$

Thus the positive branch returns to v without replenishing the tested register. The zero branch supplies the exit and its fixed increments.

Theorem 4.2 (Exact all-register drain semantics)**.** *For every register file R, a drain at v reaches w in exactly $R_r + 1$ checkpoint transitions. Its final register file $D(R)$ is*

$$D(R)_i = \begin{cases} b_r, & i = r, \\ R_i + R_r a_i + b_i, & i \neq r. \end{cases}$$

In particular, the exit increments are applied exactly once, including a possibly nonzero increment of the tested register on exit.

Proof. Put $n = R_r$ and induct on n. For $n = 0$, the zero edge is taken immediately, giving the formula and one transition. For $n + 1$, the positive edge changes the tested register to n, because $a_r = 0$, and adds a_i to every other register. Applying the induction hypothesis to this new register file yields $R_i + a_i + na_i + b_i = R_i + (n + 1)a_i + b_i$ for $i \neq r$. The tested register has value b_r at the exit. The number of transitions is $1 + (n + 1) = n + 2$. □

The checked theorem is `DrainCore.drain_exact_steps`. The supporting `drain_halts_iff` theorem proves equivalence of halting at the original configuration and the accelerated exit configuration. No bound on R_r is used. The proof also permits repeated entries in the finite increment list: their contributions are summed by the definition of `amount`.

4.1 Why this is a useful simplification

The loop proof depends on a tiny syntactic condition and one natural-number induction. It does not depend on where the loop occurs, on the numerical calculus, on the caller's tape encoding, or on a prior bound on the registers. All 1,059 occurrences can therefore use the same theorem. Their certificates check the loop shape and exit structure; they do not contain 1,059 separate arithmetical arguments.

The earlier proposal encodes these operations as homogeneous 10×10 matrices. The present kernel theorem uses the direct arithmetic formula and composition by `normalize`. Its trusted proof does not require the precomputed dictionary of 33 matrices. The matrix presentation remains a useful human-readable and synthesis representation, but the matrix dictionary's existing Python validation should not be confused with our new kernel theorem.

5 Contracting drains without losing divergence

Let $\mathrm{norm}_f(v, R)$ follow at most f recognized drains, accelerating each by Theorem 4.2. If the current vertex is not a drain, normalization stops immediately. Define the accelerated transition from any nonhalting vertex by executing *one genuine original tested edge* and then normalizing its successor. This first real transition guarantees positive progress even when there are no following drains.

Theorem 5.1 (Generic contraction correctness)**.** *For every graph G, every fuel f, every vertex v and every register file R,*

$$\mathrm{Halts}_{\mathrm{Core}_f(G)}(v, R) \quad \Longleftrightarrow \quad \mathrm{Halts}_G(v, R).$$

The analogous statement holds after normalizing the initial configuration.

Proof. For soundness, expand each accelerated edge into its original first transition and the finite drain runs. Normalization preserves halting by induction on its fuel. Composing the finitely many expanded edges gives an original halting run.

For completeness, use an original halting-time budget. Define $\mathrm{Within}_G(n, v, R)$ to mean that a halt can be reached within n transitions, allowing an already halted configuration at any budget. An original transition consumes at least one unit of a nonzero budget. Determinism, up to pointwise register equality, implies that following an actual prefix cannot increase the remaining budget. Each drain is such a finite prefix. Consequently, normalization cannot increase the budget. After one genuine tested transition and normalization, induction applies to a strictly smaller budget. This yields a finite accelerated halting derivation. □

The generic theorem is valid for any normalization fuel. A rank bound is needed to prove that the accelerated control flow stays in a chosen smaller core, not to establish the generic halting equivalence. This separation makes the proof more robust: an insufficient fuel value cannot silently certify an incorrect halting equivalence, although it may fail to establish the advertised core size.

For the concrete graph, every removed vertex has a strictly decreasing exit rank, with maximum rank six. Kernel computation checks that normalizing every defined vertex for six passes reaches the retained set. It also checks closure of every accelerated successor in that set. The retained vertices are distinct; there are 366 tests and one halt. The initial vertex 0 is already retained.

6 Binding the 367-vertex core to both literal programs

Theorem 6.1 (Literal source adequacy)**.** *For every natural-valued register file R,*

$$\mathrm{rm_halts}(\mathit{p215}, ([], R)) \iff \mathrm{core_halts}(6, G, 0, R),$$
$$\mathrm{rm_halts}(\mathit{dagPerm}, ([], R)) \iff \mathrm{core_halts}(6, G, 0, R),$$

where G is the literal checked checkpoint graph supplied in the packet.

Proof. The finite `node_ok` checks establish source instruction, branch, increment and address agreement at all 1,426 graph vertices for each literal program. Graph closure supplies all successor vertices. A generic adequacy proof converts a graph halting derivation into source segments, and conversely uses strong induction on the source halting time to peel off a tested instruction and its following increment-only segment. Address equivalence handles the implicit infinite zero suffix. Pointwise register equalities account for the effects of the segment. The checked initial addresses are equivalent to the empty path. Apply Theorem 5.1. □

The exported concrete theorems are:

```
p215_core_halts R :
  rm_halts p215 ([], R) <-> core_halts 6 k215 0%N R

dagPerm_core_halts R :
  rm_halts dagPerm ([], R) <-> core_halts 6 k215 0%N R

p215_dagPerm_affine : forall R,
  rm_halts p215 ([], R) <-> rm_halts dagPerm ([], R)
```

The last theorem has the existing source-correspondence shape. It is derived by rewriting both sides through the common core. The five new modules separate generic drain semantics, generic contraction, finite graph data, generic source adequacy, and the two concrete source bindings. The final source proof includes ten inherited prerequisite modules. No complete production Rocq binary/ZFC rebuild is inferred merely from this new source theorem.

Quantity	Checked value
Original checkpoint vertices	1,426
Recognized drain vertices	1,059
Retained core vertices	367
Retained tests / halts	366 / 1
Retained outgoing edges	732
Maximum normalization rank	6
Local drain shapes, prior analysis	24
Affine matrices, prior presentation	33

The retained control interface is approximately 25.74% of the original checkpoint set. This number is not a measured compilation-time improvement. The accelerated relation still consults the original graph data; the 367-control claim is established by the retained-set and closure checks. All removed shapes must still be checked, the graph is explicitly bound to two programs, and the generic library has a cost. For today's already equal source graphs, the earlier direct normalized certificate may remain faster and smaller. In this implementation, the generated graph/rank and location sources total about 285 KB, compared with 109,958 bytes for the earlier normalized certificate. Thus the core reduction is not a generated-byte reduction. The new result becomes more valuable when comparing semantically equivalent sources with different low-level loop decompositions.

7 Which other 215-state machine is easiest to prove?

The answer depends on the proof obligation being minimized. For the least additional work from the current certified K215 endpoint, CM215 is the strongest practical choice among the inspected proposals. For a smaller isolated marker argument, M124/FM215 is competitive. For clearer independent scheduler mathematics, the zero-credit source family is attractive, but its exact binary-machine endpoint still needs substantially more work.

Candidate	Main proof advantage	Remaining cost or qualification
CM215	Same source; six-row local transport with an existing exact dense adapter	Reuses the entire K215 calibration; smallest incremental endpoint route, not a smaller total logical foundation
M124 / FM215	Four changed rows and two local marker facts	Existing standalone transport still needs an exact repository-semantics adapter
ZC215	Occurrence-injection proof removes protected-tail credit	Concrete scheduler and exact physical implementation must still be connected end to end
R41 / rebalanced	Decoder depth decreases from 47 to 41	All 4,751 primitive occurrences and 6,176 guarded cases remain; no measured full-proof reduction
Context215	Five changed rows	Six block simulations and history conditions; row count understates proof cost
DR215 / GC-ZC215	More substantial source design changes	New parameter/selector or generalization/scheduling obligations make relative transport longer

These comparisons concern the currently available proof routes, not lower bounds on proof complexity. All listed candidates were already present in the reviewed proposals.

7.1 The exact CM215 patch

In dense labels, the existing CM215 table changes exactly the following rows. Every unlisted row is inherited from the primary K215 table.

State / read	K215	CM215
52 / 0	$0\,R\,149$	$0\,L\,170$
151 / 1	$1\,R\,170$	$1\,R\,52$
160 / 0	$0\,R\,170$	$0\,R\,52$
170 / 0	$0\,R\,163$	$0\,R\,178$
170 / 1	$1\,R\,145$	$1\,R\,125$
184 / 0	$0\,R\,170$	$0\,R\,52$

The old-run invariant has three clauses, with h the head position and t the tape:

$$q = 52 \Rightarrow t(h-1) = 1, \qquad q = 151 \Rightarrow t(h+1) = 1, \qquad (q = 170 \wedge t(h) = 0) \Rightarrow t(h-1) = 0.$$

The boundary map sends old state 170 to new state 52 and fixes every other label. Most old steps correspond to one new step. The exceptional blocks are $52 \to 170 \to 125 \to 149$ and $52 \to 170 \to 178 \to 163$, each of length three. Every simulated boundary preserves the whole tape and head position. New state 170 is still used inside these blocks, so identifying two old boundary controls does not remove a physical working state.

Because the boundary map is noninjective, its inverse cannot simply be used to prove the reverse implication. The inherited positive-stuttering theorem uses determinism and positive block lengths to synchronize any finite target halt with the source run. This is the mathematical reason the small transport proof can replace a new register-runtime and startup verification.

The dense CM215 table has SHA-256

`1f4f084f7293e2b042d5746039f793ff72ed429083b504970c939a5515e666cf.`

The parent dense table has SHA-256

`fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090.`

The exact 430 rows of both tables are independently compared with the Lean definitions. The new contribution is the completion and audit of the concrete endpoint attachment against the pinned current dependency closure; the table, invariant, transport theorem and attachment design come from the earlier `k215-transport` proposal.

8 A direct opportunity to shorten the Lean semantic proof

The earlier affine-core packet proposes a shorter proof in `KSeedModel222`. It removes three repeated formula inductions by using an already proved semantic encode/decode theorem at the model boundary. A proof that descends through every formula constructor should be written at the interpretation boundary; callers should not repeat it when they only need a special case.

The concrete change removes `enc_pos`, `enc_dec_enc` and `eqFree_dec_enc`. These were used to take a named formula through encoding, decoding, equality-freeness and re-encoding before

applying the bundle rule. Instead, construct the equality-free named bundle φ directly and prove that its interpreted code is derivable using `C222.bundle`. The existing hypothesis about a model of all derivable named formulas then gives $\mathrm{Sat}(\varphi)$. Apply the already proved semantic round trip

$$\mathrm{Sat}_{\mathrm{mem},e}(\mathrm{dec}(\mathrm{enc}(\varphi))) \iff \mathrm{Sat}_{\mathrm{mem},e}(\varphi)$$

from `Code254Models.sat_encode_decode`. Its existing irreflexivity hypothesis handles the bottom-formula alias. No new interpretation theorem or import is needed. The one remaining numerical lower bound follows from `le_P_right` and $F = 1$, rather than induction on a formula.

On the pinned source, the module decreases from 269 to 234 lines (35 lines, 13.0%) and from 11,711 to 10,436 bytes (1,275 bytes, 10.9%). The patch changes no machine table or source program. The public `seed_eqFree` theorem and the endpoint statements retain their types. The helper API does change: `derivable_bundle` becomes `derivable_named_bundle`, and the three redundant helpers are removed. Any additional consumers outside the checked endpoint closure would need to use the new interface or retain a compatibility layer.

The verification component rebuilds the exact ten affected project modules and four endpoint clients in a separate overlay after checking the declared 184-module baseline, including the four proof-only batching overlays. This makes the acceptance criterion dependent theorem compilation, not merely a text diff or an isolated tactic check. The final outcomes and assumption reports are recorded in the accompanying receipts.

This is a different kind of improvement from the graph contraction. It can reduce maintained handwritten source directly, while the graph construction adds generic infrastructure and checked generated data. Both are useful, but their measurements should not be mixed. In particular, lines removed from one module do not establish a proportional improvement in total endpoint build time.

9 Research toward 214 states

9.1 What was searched

We explored new local quotient shapes in addition to the existing same-head, same-tape folds: identifying states with a head displacement, and identifying them while toggling a nearby marker cell. The scripts record their finite window, step bounds and chosen invariant. Their failures only delimit these specific proof/search families.

The main new experiment addresses the most promising relaxation suggested by the automatic-certificates packet. The register runtime's logical interface uses nine registers, a continuation, a legal gap, and a nonempty spare suffix. The high-level representation need not expose the exact spare suffix returned by each primitive. Requiring an exact whole-tape join can therefore reject transformations that preserve every logical register and continuation.

Our relaxed decoder accepts any valid nonempty spare suffix and legal gap, while requiring the nine logical registers and continuation to be exact. The search deletes one state, rewrites its unique external incoming row, and tests the new machine on valid primitive frames. For the main survey, 29 covered labels, 214 replacement targets, two writes and two directions give

$$29 \cdot 214 \cdot 2 \cdot 2 = 24,824$$

candidates. The initial screening cap is 5,000 transitions. It is a bounded screen, not a decision procedure for arbitrary candidate correctness.

9.2 Counterexamples for all short-screen survivors

Fifty-nine candidates survive the first short opcode-class tests. They are then tested at actual complete-source primitive occurrences. Fifty have an explicit wrong decoded logical result at a clean dispatcher. The other nine initially exceeded a 20,000-transition checking cap. A separate cycle analysis then found literal repeated complete configurations, with zero head displacement, by steps 585–609. No halt or successful required redispatch lies between each recorded repeat. Determinism therefore proves infinite repetition on those particular valid primitive inputs.

An independent byte-array simulator checks all 59 witnesses, the corresponding parent frames, the expected source effects, and the claimed first redispatch times. It does not import the search implementation. For each cycle it compares the state, head and every nonzero tape cell. The finite witness plus determinism establishes the claimed loop without extrapolating from a timeout.

These are counterexamples to the uniform relaxed primitive contract that requires the correct result at the first clean redispatch. They do not exclude a different simulation relation that permits extra intervening redispatches. They do not prove that every mutated blank-start machine has a different consistency behavior. Nor do they turn every one of the earlier bounded-screen failures into a complete mathematical refutation. The justified conclusion is that this pipeline produced no successful reduction and that its 59 initially promising survivors all fail the intended reusable simulation obligation.

9.3 A concrete shared-helper obstruction

Dense state 189 is reached externally from row 66/0. In the inspected implementation it participates both in spare extension for increments and in restoration on a zero decrement. A replacement that looks correct when the increment is allowed to leave a different spare suffix can still destroy the decrement restoration route. This is why weakening the spare-output condition alone does not automatically free a state: shared physical control has more than one logical role.

There is a positive, quantified sub-contract. Change the native row

$$66/0: \quad 1\,R\,198 \quad \longmapsto \quad 0\,L\,54$$

and delete native state 198 (dense state 189). The resulting research table has 214 listed working labels and 428 rows; 212 working labels remain reachable in its transition graph. The inherited symbolic runtime checker proves all 1,900 literal increment occurrences with the exact logical increment and continuation, while leaving the arbitrary nonempty spare suffix unchanged. Its induction rules cover arbitrary natural register values, arbitrary natural gap and arbitrary finite spare data; this is stronger than bounded testing. The run uses 672,539 symbolic rules. It is an ordinary quantified symbolic argument checked by a Python verifier, not a proof-assistant kernel theorem.

The same table fails a concrete zero-decrement contract. At zero-based source index 2 (the third leaf), address `0100`, command `CD0`, take all nine registers zero, spare suffix [0] and gap zero. The parent returns after 640 transitions with all logical registers zero and continuation one-bits at positions $[1, 2]$. The research table first returns after 3,966 transitions with continuation one-bits at $[1, 3, 4, 5]$ and register values

$$[0, 0, 0, 0, 0, 1, 0, 0, 2].$$

An independent simulator replays this mismatch. Thus the 214-state table is delivered only as a rejected uniform-runtime experiment, with its failed obligation prominently identified. It is not delivered as a verified solution to the consistency machine problem; its blank-run calibration remains unestablished.

All 856 direct write/direction/target replacements for the incoming row also fail the stated relaxed short screen. A targeted 856-choice provider experiment likewise finds no sampled full-contract survivor. These are bounded observations, not global impossibility results for a joint rewrite of the shared region.

9.4 Coordinated reuse of an existing provider

A further fixed family redirects dense row 66/0 to another provider p and copies either or both outgoing rows of state 189 into p, then deletes 189. Excluding providers 66 and 189 gives $213 \cdot 3 = 639$ tables. All 109 survivors of the short screen fail the subsequent complete-source sample refinement. An independent simulator audits the whole family, with the following results.

Observed obstruction or remaining bound	Tables
Wrong first-redispatch output	338
Outward motion into the infinite blank exterior	223
Literal repeated configuration	5
Unexpected halt during a nonhalting primitive	2
Unresolved at the 20,000-transition bound	71

Thus 568 tables have explicit primitive-contract obstructions; the remaining 71 have only bounded failures. The ray checks verify a read-zero self-loop moving away from every nonzero cell, and the cycle checks compare complete configurations. These conclusions are stronger than timeouts but remain specific to the stated primitive contract.

Three variants, using providers 95, 188 and 213 with both rows replaced, are particularly informative. Each has 214 listed and graph-reachable working states. Their first decoded blank-start frames match the parent's complete tape and head: at 79,633 transitions for the first two, and 79,639 for the third. This experimental first decoded frame is distinct from the maintained formal bootstrap endpoint. They pass respectively 5,944, 5,936 and 5,918 guarded source cases before their first failed case. None of those finite successes establishes correctness.

Their failures occur at zero-based source indices 4,578, 4,572 and 4,560. The bad prefixes read only positions -10 through 14, 15 or 21 relative to the initial head. Every input cell depending on registers, gap or spare data lies at position at least 64. The returned continuation differs from both branches of the source decrement. The same finite prefix therefore fails for every natural register file and every legal gap and spare suffix at that address. A rescue by a stronger input invariant must exclude the address itself; excluding only the displayed zero-register valuation is insufficient. The independently replayed tables and witnesses are retained as rejected experiments in `state_search/forced_provider189/`.

The addresses also have a useful source-level explanation. They lie in the mandatory final gate of the AX routine: the first comparison, restoration of its saved value, and the second comparison. Direct graph replay reaches a `Clean` AX entry after 180 checkpoint transitions from the supplied initial decoded frame. The inherited total macro-run theorems then reach the gate with registers $[7, T, W, 0, 0, 0, 0, 0, 0]$ for finite T, W. The literal graph visits source indices 4,560, 4,572 and 4,578 at offsets 1, $T + 4$ and $T + 6$ from gate entry. Thus an ordinary argument from inherited totality and the checked literal mapping establishes their eventual occurrence in the parent source run; a long bounded trace was merely still doing earlier arithmetic. This argument is documented in `state_search/source_reachability/`. It is not a new kernel reachability theorem, and it does not establish that any altered physical machine follows the parent to those controls or has a different blank-tape halting behavior.

The overwritten providers are source-address decoder controls. Provider 188 is especially localized: it separates two closed transfer loops, from register 7 to 6 and from register 4 to 5. Three simple swaps or reassociations of this region preserve 202 distinct source DAG nodes; the corresponding literal two-node rewires each still have 215 working states and no ordinary Mealy-state merge. The source transformations are justified by closed-block sequencing and the disjoint register pairs. They do not certify those physical rewires: a changed intermediate decoder context still needs its own runtime proof. This small experiment saves no state. A more substantial joint implementation of the two transfers, their decoder and the shared append/restoration interface remains a concrete synthesis target.

Subsequent synthesis should require the increment and zero-decrement obligations together from the beginning. Source equivalence, exact decoder binding, quantified primitive contracts and positive simulation progress are separate acceptance gates. Validating only an increment subcase and counting a deleted shared state does not establish the desired calibration.

10 Integration plan

The following order makes each change reviewable and keeps the exact existing endpoint available throughout.

1. Apply the minimal build-driver correction if using the supplied current Lean driver. Its compiler identity query should use quiet Lake output: one-time manifest diagnostics otherwise change the captured identity string even when the compiler binary and all hashes are unchanged. The packet retains the reproduced failure, the one-line patch, and the resumed-build check. Apply the separately disclosed four certificate-proof batching patches when using the recorded memory-limited build, together with their checked generator postpass so regeneration preserves the proof structure.

2. Integrate the checked `KSeedModel222` source patch and its dependent theorem checks, subject to the precise outcome in the included verification record. The patch changes no generator or machine table.

3. Add CM215's already designed exact-table adapter and endpoint with the completed verification receipts. Keep the six changed rows, invariant clauses, literal-table bindings and attribution together.

4. Add the affine-core files as a separate checked source lane. Its exact public source-equivalence theorem can be used behind a same-statement facade. Do not remove the existing normalized certificate merely because the retained core has fewer vertices. Measure clean compilation and review both proof sizes before choosing a maintained default.

5. Use the core interface for new source designs, while keeping binary allocation and quantified runtime verification as separate gates. A successful semantic source simplification is a proposal for physical synthesis, not an automatic state saving.

The archive is additive and does not modify the remote repository. It preserves the source inputs needed for offline reproduction, compiler logs and small patches. Compiled proof objects and toolchain binaries are intentionally not part of the deliverable; the build scripts reconstruct the relevant objects from the supplied sources.

11 Further research questions

1. **Can different source variants share a proved arithmetic core?** The present two programs already have the same checkpoint graph. Apply the new interface to zero-credit, transfer, or rebalanced sources whose low-level graphs differ. A successful common-core certificate would demonstrate the benefit that motivates this additional infrastructure.
2. **Can more total loop patterns be recognized soundly?** The drain rule requires no invariant beyond its syntax. Useful extensions include paired copy-and-restore loops, swaps, and guarded affine blocks. Every extension should carry an exact termination measure and should preserve the explicit decrement at a retained test.
3. **Can a verified coefficient representation make the data smaller?** The direct arithmetic normalizer avoids trusting the prior matrix dictionary. A kernel-checked sparse matrix evaluator with a shared dictionary could reduce duplicated arithmetic data, provided measurements justify the added layer.
4. **Can M124/FM215 receive the same exact endpoint attachment?** Its local marker argument is smaller than CM215's. A short adapter into the repository's dense semantics may make it the preferable next independently attached 215-state variant.
5. **Can the increment and zero-decrement helper be resynthesized together under existential spare output?** The concrete failures identify a shared restoration obligation that a local increment-only optimization misses. This joint region is a more informative target than another broad same-tape state-fold census.
6. **Can equiconsistency simplify the source enough to survive allocation?** The quotient-detachment result demonstrates a mathematically valid relaxation of the numerical calculus, but its first physical implementations cost more states. Search objectives must combine arithmetic proof simplicity, address sharing, command selection, spare management and physical return routing.
7. **Which simplifications reduce total verification cost?** Measure handwritten proof lines, generated bytes, kernel memory, clean-build time and independent checking separately. A single headline reduction in certificate entries or graph vertices does not answer all five questions.

12 Reproduction and provenance

The top-level README provides exact build commands for each component. The manifest binds every delivered payload except the manifest itself. A manifest check establishes file identity; it is not a semantic theorem. The default portable verifier also replays the independent search witnesses. The Coq and Lean build commands are separate so that a Python integrity check cannot be mistaken for a kernel check.

The mathematical calibration, primary binary table, literal register programs, segment semantics, normalized-address tools, CM215 design, and incoming three-induction proposal are inherited. New contributions are the checked drain and contraction library, the generic checked graph/source bridge and its exact two-program instantiation, the completed verification work explicitly recorded in this packet, the minimal driver fix, and the new local-search experiments and independent counterexample audit. The packet does not claim a new logical independence theorem, a new proof of ZFC consistency, a minimum-state result, or a verified 214-state machine.

13 Primary sources

All repository links below are pinned to the reviewed revision. The original files and packet notices identify their individual contributors and inherited licenses; the archive retains the relevant notices. References to the new proofs refer to source files in this archive and their final verification receipts.

[1] ProveIt, primary K215 description and exact machine convention. `candidate-k215/README.md`.

[2] ProveIt, exact Lean endpoint, `KEndpoint215.lean`.

[3] ProveIt, current simplification and proposal index, `simplification/proposals/README.md`.

[4] ProveIt, prior affine-core proposal, `proposals/affine-cores/`.

[5] ProveIt, existing local transport and endpoint attachment sources. `proposals/k215-transport/`.

[6] ProveIt, automatic certificates and concrete credit proofs. `proposals/automatic-certificates/`.

[7] ProveIt, alternative-machine index, `candidate-k215/alternatives/README.md`.

[8] ProveIt, incoming quotient-detachment packet. Quotient-detachment research archive.

[9] ProveIt, checked framed-decoder foundation and remaining attachment obligations. Checked decoder library, and the precise leaf interface in runtime proof factoring.